Java Backend Architecture

Interview Series

Chapter 11.2
Pessimistic Locking

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

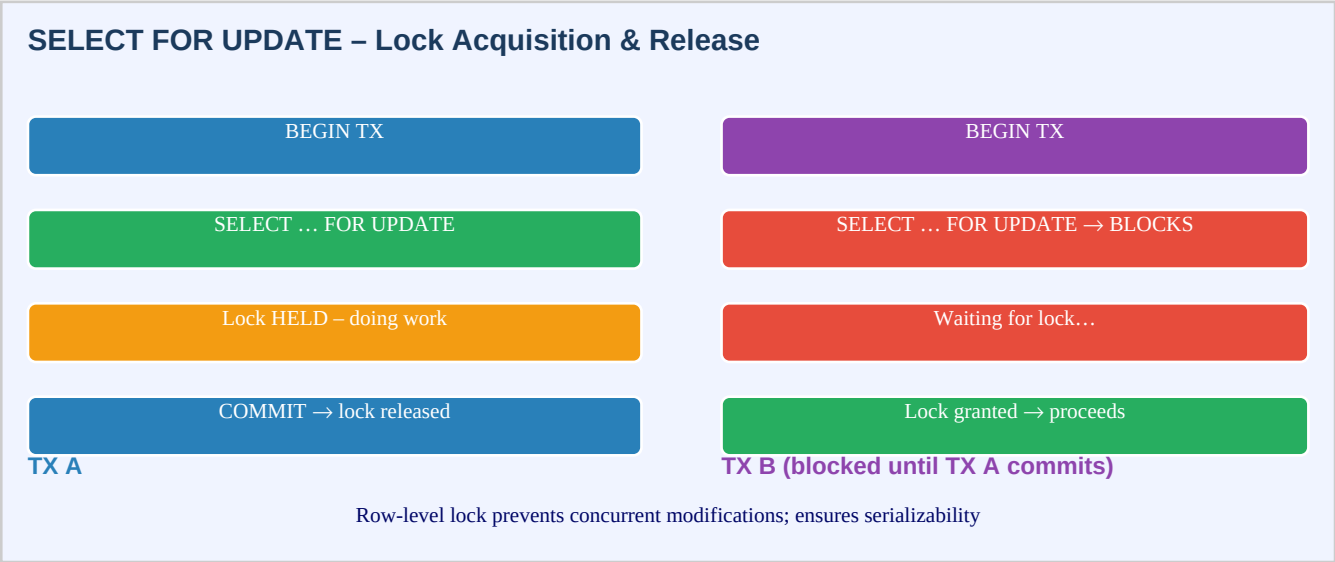
Chapter 11.2 – Pessimistic Locking

Introduction

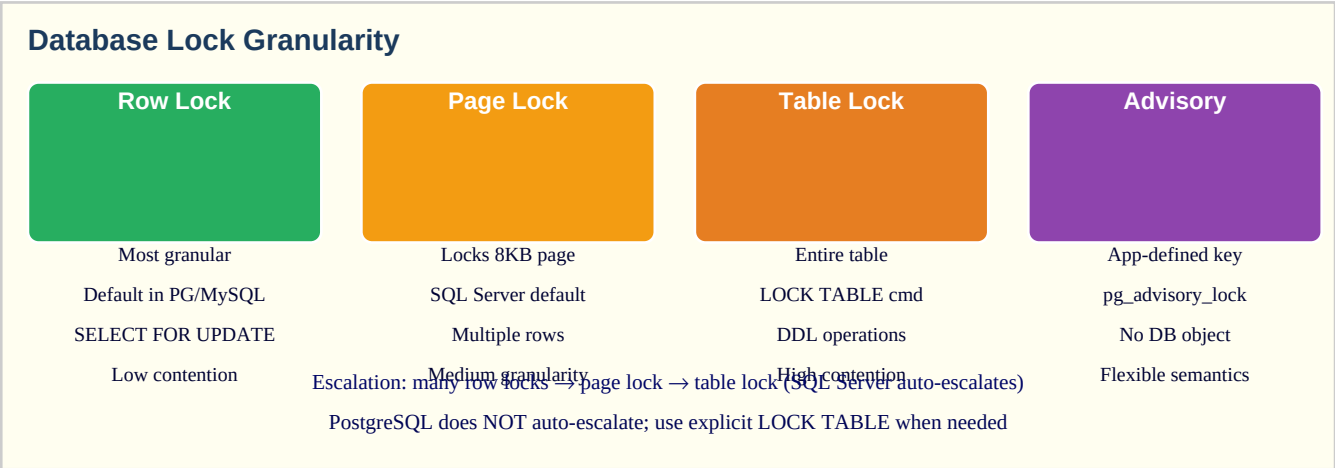
Pessimistic locking assumes conflicts will occur and prevents them by acquiring an exclusive lock before reading data that will be modified. The canonical SQL mechanism is SELECT FOR UPDATE, which acquires a row-level exclusive lock at SELECT time and holds it until the transaction commits or rolls back. Other transactions that attempt to select the same rows FOR UPDATE will block until the lock is released.

Pessimistic locking is appropriate when conflicts are frequent and the cost of retrying is high. It serializes writers cleanly but reduces concurrency. The two key failure modes are deadlocks (two transactions waiting for each other) and long-held locks (a slow transaction blocking all other writers). Both are mitigated with lock timeouts and consistent lock-ordering.

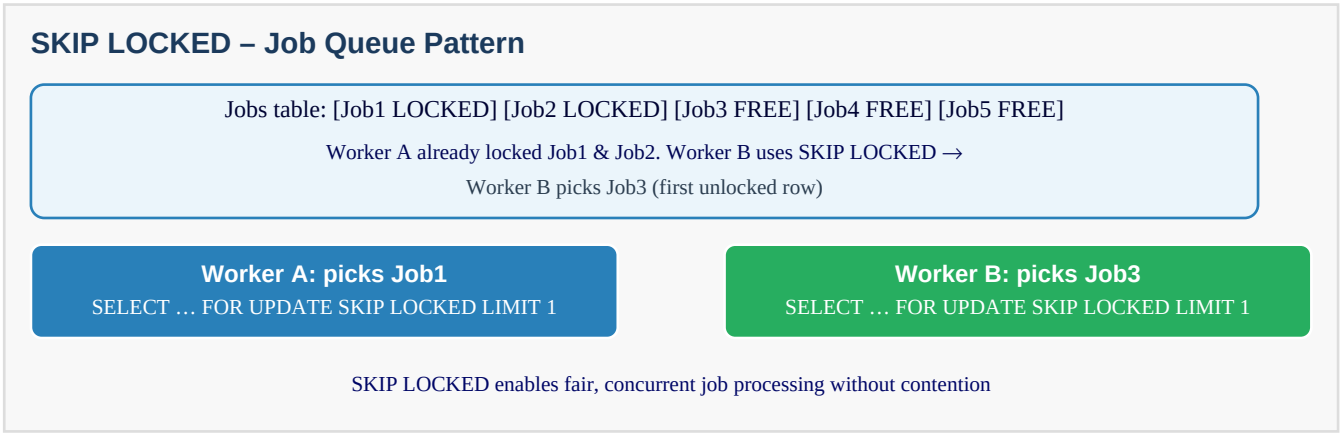
SELECT FOR UPDATE – Lock Lifecycle



Lock Granularity



SKIP LOCKED Job Queue Pattern



Key Definitions

SELECT FOR UPDATE	SQL clause that acquires an exclusive row lock on selected rows
SELECT FOR SHARE	Acquires a shared lock — other readers can share, writers block
NOWAIT	Return error immediately if lock is unavailable instead of waiting
SKIP LOCKED	Skip rows that are locked by other transactions — for queue processing
lock_timeout	Max time to wait for a lock before raising LockNotAvailableException
LockModeType.PESSIMISTIC_WRITE	JPA lock mode mapping to SELECT FOR UPDATE
LockModeType.PESSIMISTIC_READ	JPA lock mode mapping to SELECT FOR SHARE
deadlock	Two transactions waiting for each other's locks — DB kills one TX
lock escalation	DB upgrades many row locks to a table lock (SQL Server, not PG)
advisory lock	Application-defined lock with a numeric key; not tied to a DB row/table
pg_advisory_lock	PostgreSQL blocking advisory lock by bigint key
pg_try_advisory_lock	Non-blocking advisory lock — returns false if already locked
deadlock_timeout	PostgreSQL: time before deadlock detection runs (default 1s)
innodb_lock_wait_timeout	MySQL: seconds to wait for row lock before giving up

Lock Mode Quick Reference

JPA LockModeType	SQL Generated	Blocks	Use Case
PESSIMISTIC_WRITE	SELECT … FOR UPDATE	Writers & readers (FOR SHARE)	Exclusive modify
PESSIMISTIC_READ	SELECT … FOR SHARE	Writers only	Prevent writes, allow reads
PESSIMISTIC_WRITE + NOWAIT	SELECT … FOR UPDATE NOWAIT	Immediate exception	Non-blocking exclusive lock

OPTIMISTIC	(no SQL lock)	Nothing	Version-check at commit

Code Examples

1. Spring Data JPA — PESSIMISTIC_WRITE

```
public interface AccountRepository extends JpaRepository<Account, Long> {
@Lock(LockModeType.PESSIMISTIC_WRITE) @Query("SELECT a FROM Account a WHERE a.id = :id")
Optional<Account> findByIdForUpdate(@Param("id") Long id); // With NOWAIT (PostgreSQL / Oracle)
@Lock(LockModeType.PESSIMISTIC_WRITE) @QueryHints(@QueryHint(name =
"javax.persistence.lock.timeout", value = "0")) // 0 = NOWAIT, -2 = SKIP LOCKED @Query("SELECT
a FROM Account a WHERE a.id = :id") Optional<Account> findByIdForUpdateNowait(@Param("id") Long
id); } @Service @Transactional public class TransferService { public void transfer(Long fromId,
Long toId, BigDecimal amount) { // Always lock in consistent order to prevent deadlock Long
lowerid = Math.min(fromId, toId); Long higherid = Math.max(fromId, toId); Account first =
accountRepo.findByIdForUpdate(lowerid).orElseThrow(); Account second =
accountRepo.findByIdForUpdate(higherid).orElseThrow(); Account from =
first.getId().equals(fromId) ? first : second; Account to = first.getId().equals(toId) ? first
: second; if (from.getBalance().compareTo(amount) < 0) { throw new
InsufficientFundsException(); } from.setBalance(from.getBalance().subtract(amount));
to.setBalance(to.getBalance().add(amount)); // Both rows locked — safe to write, locks released
on TX commit } }
```

2. Raw SQL — lock_timeout per session

```
-- PostgreSQL: set lock timeout for the current transaction SET LOCAL lock_timeout = '3s'; --
throw if lock not obtained within 3 seconds SELECT * FROM inventory WHERE product_id = 42 FOR
UPDATE; -- MySQL: global or session innodb_lock_wait_timeout SET SESSION
innodb_lock_wait_timeout = 5; SELECT * FROM inventory WHERE product_id = 42 FOR UPDATE; --
NOWAIT: fail immediately if locked SELECT * FROM orders WHERE id = 100 FOR UPDATE NOWAIT; --
SKIP LOCKED: job queue SELECT id, payload FROM job_queue WHERE status = 'PENDING' ORDER BY
priority DESC LIMIT 1 FOR UPDATE SKIP LOCKED; -- Only returns rows not currently locked by
another session
```

3. JDBC – lock timeout hint

```
// JPA EntityManager with lock hints Map<String, Object> hints = new HashMap<>();
hints.put("javax.persistence.lock.timeout", 3000); // 3 seconds in ms Product product =
entityManager.find( Product.class, productId, LockModeType.PESSIMISTIC_WRITE, hints ); // If
lock not obtained within 3s → LockTimeoutException // Catch and return HTTP 409 or retry try {
Product p = entityManager.find(Product.class, id, LockModeType.PESSIMISTIC_WRITE, hints); //
... do work } catch (LockTimeoutException e) { throw new ResourceLockedException("Product is
being updated by another request"); } catch (PessimisticLockException e) { throw new
ConcurrencyException("Could not acquire lock", e); }
```

4. SKIP LOCKED — job queue in Java

```
@Repository public interface JobRepository extends JpaRepository<Job, Long> {
@Lock(LockModeType.PESSIMISTIC_WRITE) @QueryHints(@QueryHint(name =
"javax.persistence.lock.timeout", value = "-2")) // -2 = SKIP LOCKED hint for Hibernate
@Query("SELECT j FROM Job j WHERE j.status = 'PENDING' ORDER BY j.priority DESC") List<Job>
findNextJob(Pageable pageable); } @Service public class JobProcessor { @Scheduled(fixedDelay =
1000) @Transactional public void processNext() { List<Job> jobs =
jobRepo.findNextJob(PageRequest.of(0, 1)); if (jobs.isEmpty()) return; Job job = jobs.get(0);
```

```
job.setStatus("PROCESSING"); jobRepo.save(job); try { doWork(job); job.setStatus("DONE"); }
catch (Exception e) { job.setStatus("FAILED"); job.setError(e.getMessage()); }
jobRepo.save(job); // TX commits → lock on this job row released // Other workers can proceed
with remaining PENDING jobs } }
```

5. PostgreSQL advisory locks

```
-- Session-level advisory lock (released on disconnect or explicit unlock) SELECT
pg_advisory_lock(12345); -- blocking SELECT pg_try_advisory_lock(12345); -- non-blocking:
true/false SELECT pg_advisory_unlock(12345); -- Transaction-level advisory lock (auto-released
on COMMIT/ROLLBACK) SELECT pg_advisory_xact_lock(12345); -- blocking SELECT
pg_try_advisory_xact_lock(12345);-- non-blocking -- Java with Spring JDBC: @Transactional
public void processWithAdvisoryLock(Long resourceId) { Boolean locked = jdbc.queryForObject(
"SELECT pg_try_advisory_xact_lock(?)", Boolean.class, resourceId); if
(!Boolean.TRUE.equals(locked)) { throw new ResourceLockedException("Resource " + resourceId + "
is locked"); } // Perform exclusive work — lock auto-released when TX commits
doExclusiveWork(resourceId); }
```

Interview Questions & Answers

Q1. What is SELECT FOR UPDATE and what lock does it acquire?

SELECT FOR UPDATE acquires an exclusive row-level lock on each row returned by the query. Other transactions attempting to SELECT FOR UPDATE or modify the same rows will block until the locking transaction commits or rolls back. Transactions that only SELECT (without FOR UPDATE or FOR SHARE) are not blocked in most databases (MVCC allows readers to see a snapshot). The lock is held for the entire duration of the transaction — not just the SELECT statement.

Q2. What is the difference between FOR UPDATE and FOR SHARE?

FOR UPDATE (exclusive lock): blocks both other writers AND other FOR SHARE readers. Use when you intend to modify the row. FOR SHARE (shared lock): allows multiple transactions to hold FOR SHARE simultaneously, but blocks transactions that want FOR UPDATE. Use when you want to prevent writes but allow concurrent reads. In JPA: FOR UPDATE = PESSIMISTIC_WRITE, FOR SHARE = PESSIMISTIC_READ. In PostgreSQL: FOR NO KEY UPDATE and FOR KEY SHARE are additional granular variants.

Q3. How do you set a lock timeout to prevent indefinite blocking?

PostgreSQL: `SET LOCAL lock_timeout = '3s'` in the transaction before the FOR UPDATE. Throws LockNotAvailableException after timeout. MySQL InnoDB: `SET SESSION innodb_lock_wait_timeout = 5`. JPA hint: `javax.persistence.lock.timeout` in milliseconds (0 = NOWAIT, -2 = SKIP LOCKED). Always set lock timeouts in production to prevent a slow transaction from blocking all writers indefinitely. The appropriate timeout depends on the expected lock duration — typically 1-10s.

Q4. What is SKIP LOCKED and what problem does it solve?

SKIP LOCKED returns rows that are NOT currently locked, silently skipping locked ones. This enables efficient work queues: multiple worker processes can each SELECT FOR UPDATE SKIP LOCKED and independently pick up different unlocked rows without waiting for each other. Without SKIP LOCKED, all workers would queue up on the same row. Each worker processes its own row concurrently, and the row is implicitly reserved (locked) for the duration of its processing transaction.

Q5. How do deadlocks occur and how does PostgreSQL handle them?

A deadlock occurs when transaction A holds a lock that B needs, and B holds a lock that A needs — circular wait. PostgreSQL runs a deadlock detection algorithm when a lock wait exceeds deadlock_timeout (default 1s). It detects the cycle in the wait-for graph and kills one of the transactions (the 'victim'), throwing ERROR 40P01 (deadlock detected). The application must catch this error and retry the transaction. MySQL similarly detects deadlocks and rolls back the transaction with the smallest undo log.

Q6. What is the most important rule for preventing deadlocks with pessimistic locking?

Always acquire multiple locks in a consistent global order. Example: when transferring between accounts A and B, always lock the lower account ID first, then the higher — regardless of the direction of the transfer. If TX1 locks A then B, and TX2 locks B then A, deadlock. If both lock in ascending ID order, they serialize naturally. Other strategies: acquire all needed locks at once (rather than incrementally), use lock timeouts to break deadlocks before the DB detects them, keep transactions short.

Q7. What is the LockModeType hierarchy in JPA?

LockModeType.NONE: no lock. OPTIMISTIC: version check at commit (same as @Version default). OPTIMISTIC_FORCE_INCREMENT: increment version regardless. PESSIMISTIC_READ: shared lock (SELECT FOR SHARE). PESSIMISTIC_WRITE: exclusive lock (SELECT FOR UPDATE). PESSIMISTIC_FORCE_INCREMENT: exclusive lock + version increment. Pass LockModeType to EntityManager.find(), Query.setLockMode(), or via @Lock on repository methods.

Q8. What happens to a pessimistic lock when the transaction rolls back?

All locks acquired during the transaction are released immediately on ROLLBACK, just as on COMMIT. Any waiting transactions can then proceed. This is a key safety property: locks are always tied to a transaction lifecycle. There is no concept of a pessimistic lock surviving a transaction failure. If the application crashes (not just a rollback), the DB detects the dead connection and releases all associated locks.

Q9. Compare advisory locks to regular row locks.

Row locks are tied to specific rows/tables in the database. Advisory locks are application-defined bigint keys with no database object attached. Use advisory locks for: (1) coordinating access to external resources (not a DB row), (2) global singleton operations (run only one instance), (3) distributed locking with PostgreSQL as the lock server. Session-level advisory locks persist until explicitly released or the session disconnects. Transaction-level advisory locks are released at COMMIT/ROLLBACK — prefer these to avoid forgotten lock leaks.

Q10. What is a 'SELECT FOR UPDATE NOWAIT' and when would you use it?

NOWAIT causes the SELECT FOR UPDATE to fail immediately (raise an exception) if any requested row is already locked, rather than blocking. Use it in interactive UIs or API endpoints where blocking for seconds is unacceptable: show the user 'This record is being edited by another user' immediately. In JPA: set `javax.persistence.lock.timeout` hint to 0. In the API layer, catch LockTimeoutException and return HTTP 423 (Locked) or 409 (Conflict).

Q11. What is the difference between a pessimistic lock at the application level vs the DB level?

DB-level (SELECT FOR UPDATE): lock managed by the database, enforced across all connections, released on COMMIT/ROLLBACK, survives application restarts within the same session. Application-level: ReentrantLock / synchronized in Java — only protects within one JVM instance; does NOT protect against concurrent access from other service instances or DB connections. For distributed systems, application-level Java locks are insufficient — use SELECT FOR UPDATE or a distributed lock (Redis, ZooKeeper).

Q12. How do you detect that a LockTimeoutException occurred and what HTTP status should you return?

Catch `javax.persistence.LockTimeoutException` (JPA) or `org.springframework.dao.CannotAcquireLockException` (Spring wrapper). Return HTTP 409 Conflict or HTTP 423 Locked (RFC 4918). Include a Retry-After header suggesting when to try again. Log the event with the resource ID and the holder's transaction info (from pg_stat_activity) for debugging. If lock timeouts are frequent, it signals long-running transactions or insufficient lock_timeout tuning.

Q13. What is lock escalation and does PostgreSQL do it?

Lock escalation: a database automatically upgrades many fine-grained row locks to a coarser table lock to save lock manager memory (common in SQL Server). PostgreSQL does NOT auto-escalate row locks to table locks — each row lock is independent regardless of count. PostgreSQL's lock manager stores lock information in shared memory; if the lock table fills up, the server will report out-of-shared-memory errors rather than escalating. Size the max_locks_per_transaction parameter appropriately for high-lock-count workloads.

Q14. How do you implement a database-backed job queue with pessimistic locking?

Schema: jobs(id, status, payload, created_at). Status: PENDING→PROCESSING→DONE/FAILED. Worker: within a transaction, SELECT id FROM jobs WHERE status='PENDING' ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 1; UPDATE jobs SET status='PROCESSING' WHERE id=?; COMMIT. Process the job outside the lock-acquisition transaction. Mark DONE or FAILED in a follow-up transaction. Benefits: atomic job reservation, no message broker needed, exactly-once processing per job (if idempotent). This is the pattern used by tools like Sidekiq, GoodJob, and Que.

Q15. What is the risk of holding pessimistic locks across slow I/O operations?

If a transaction acquires a SELECT FOR UPDATE and then calls an external API, sends an email, or does disk I/O before committing, the row lock is held for the entire duration. Other transactions that need the same rows block — potentially for seconds or minutes. Best practice: perform all slow operations BEFORE the lock transaction, or move them AFTER the COMMIT. The lock transaction should only: read locked data, make decisions, write, commit. Minimize time between lock acquisition and release.

Q16. How do you test pessimistic locking behavior in a Spring Boot integration test?

Use two threads with separate transactions. Thread 1: begin TX, SELECT FOR UPDATE on row R, sleep 2s (simulating slow work), commit. Thread 2: begin TX, immediately SELECT FOR UPDATE on R with lock_timeout=500ms. Thread 2 should either block for ~2s (then succeed) or throw LockTimeoutException. Use CompletableFuture or CountDownLatch to coordinate. Verify via database state that writes are serialized. In-memory H2 supports row locking in MODE=PostgreSQL.

Q17. What is a phantom read and does SELECT FOR UPDATE prevent it?

A phantom read occurs when a transaction re-executes a range query and sees new rows inserted by another committed transaction (rows that didn't exist in the first read). SELECT FOR UPDATE locks existing rows but does NOT lock the gaps between rows (in most DBs). New inserts in the range can still appear. To prevent phantom reads: use SERIALIZABLE isolation level (PostgreSQL SSI or predicate locking). In PostgreSQL, REPEATABLE READ prevents phantoms via MVCC snapshot (unlike SQL standard definition).

Q18. How does Spring's @Transactional interact with pessimistic locking?

Pessimistic locks are transaction-scoped — the SELECT FOR UPDATE must be inside a @Transactional method. The lock is acquired when the SELECT executes and released when the method returns (transaction commits/rolls back). Calling a @Lock repository method without @Transactional on the caller causes Spring Data to open a transaction just for the query, commit immediately, and release the lock — completely useless. Ensure the ENTIRE lock-work-update sequence is within one @Transactional boundary.

Q19. What is a 'hot row' problem with pessimistic locking?

A hot row is a frequently-updated row that becomes a serialization bottleneck under pessimistic locking. Example: a global inventory counter that every order decrements. All order transactions queue on the same row's lock, creating a throughput ceiling of one write per lock duration. Solutions: (1) Sharding the counter (multiple partial counters, sum for reads). (2) Optimistic locking with retry (if conflict rate is acceptable). (3) Asynchronous counter update via queue (decouple the order from the inventory decrement). (4) Redis INCR/DECR for in-memory counters with async DB sync.

Q20. Explain the difference between lock_timeout and statement_timeout in PostgreSQL.

`lock_timeout`: maximum time to wait to acquire a lock; throws LockNotAvailableException. Only applies to lock acquisition, not query execution. `statement_timeout`: maximum total time a statement can run; throws QueryCanceled. Applies to the entire statement execution including lock wait. For pessimistic locking: set lock_timeout to control lock wait specifically without interfering with legitimate long-running queries. Set both for defense-in-depth. Set per session (`SET LOCAL`) to scope to the current transaction only.